

NOVAC Internship – Task 1 Daily Documentation

Task: Learning Web APIs using Flask and FastAPI

Overview

Today's task was focused on understanding the fundamentals of Web APIs and implementing basic API calls using both Flask and FastAPI. The session started with learning the theory behind APIs, REST architecture, HTTP methods, request-response cycles, and frontend-backend communication. After understanding the concepts, small prototype applications were built using Flask and FastAPI along with a simple HTML frontend.

Resources and References Used

- Microsoft Learn – REST APIs in ASP.NET Core
- FastAPI Official Documentation
- Flask Official Documentation

Concepts Learned

What is an API: An API (Application Programming Interface) acts as a communication bridge between two software systems. It allows a frontend application to send requests and receive responses from a backend server.

Web APIs: A Web API uses HTTP protocols to enable communication over the internet or local network. Responses are usually returned in JSON format.

REST APIs: REST APIs follow standard HTTP methods such as GET and POST and are designed to be lightweight, scalable, and modular.

GET Requests: GET requests are mainly used to retrieve or fetch data from the backend. In the prototype project, clicking the GET button fetched a response message from the server.

POST Requests: POST requests are used to send data from the frontend to the backend. In the prototype, the entered username was sent to the backend and the backend returned a dynamic response.

JSON: JSON (JavaScript Object Notation) is a lightweight format used for exchanging structured data between frontend and backend systems.

Frontend-Backend Communication: JavaScript `fetch()` was used to send GET and POST requests from the HTML frontend to the Flask/FastAPI backend.

Libraries, Modules, and Components Learned

Flask: Used to create lightweight web applications and APIs.

jsonify: Converts Python dictionaries into JSON responses.

request: Used to access incoming request data from the client.

render_template: Used to render HTML files from the templates folder.

FastAPI: Modern Python framework used for building fast and structured APIs.

Request: Represents incoming HTTP requests in FastAPI.

HTMLResponse: Used to return HTML content from FastAPI routes.

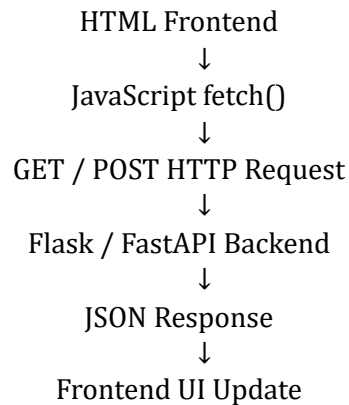
BaseModel: Provided by Pydantic and used for request validation and schema creation.

Practical Implementation

- Installed Flask, FastAPI, and Uvicorn.
- Created a Flask API with GET and POST endpoints.

- Created a FastAPI application with GET and POST endpoints.
- Built simple HTML frontend pages for both frameworks.
- Connected frontend buttons with backend APIs using JavaScript fetch().
- Tested API responses on localhost using ports 5000 and 8000.
- Sent JSON data from frontend to backend using POST requests.
- Displayed backend responses dynamically on the webpage.
- Debugged template and file path related errors.

Architecture Understood



Challenges Faced

While implementing the project, template-related errors such as `TemplateNotFound` and `FileNotFoundError` were encountered. These issues helped in understanding Flask's template folder structure and FastAPI file handling. Additional debugging involved restarting the server, checking file paths, and understanding terminal traceback logs.

Conclusion

Today's task helped build a strong beginner-level understanding of backend development and API communication. By the end of the session, working prototypes using Flask and FastAPI were successfully created with functional GET and POST requests connected to a frontend interface.